📋 Analyse du Cahier des Charges — Application RH Logicat

🎯 Vue d'ensemble du projet

Application RH complète avec deux espaces distincts :

- **Administrateur RH** — gestion des utilisateurs, suivi des pointages/pauses, gestion des tâches
- **Employé** — profil personnel, pointage quotidien (check-in/out + 3 pauses), consultation/mise à jour des tâches

Durée recommandée : 5 jours | **Format** : Projet individuel

🔐 1. Authentification & Sécurité

Concept	Détails
Authentification	Email + mot de passe → JWT token
Hachage	Mots de passe **jamais** en clair (bcrypt/argon2)
2 rôles	`Admin RH` et `Employé`
Route protection	Côté **serveur** obligatoire (pas juste frontend)
Compte désactivé	Doit être bloqué même avec un JWT valide
Pas d'inscription publique	Seul un Admin RH peut créer des comptes
Admin initial	Un compte Admin RH seed pour démarrer

> IMPORTANT
>
> **Règle critique** : Un Admin RH ne peut **ni** désactiver son propre compte, **ni** le dernier Admin RH actif du système.

👨‍💼 2. Gestion des utilisateurs (Admin RH uniquement)

Champs requis

Catégorie	Champs
Identité	Nom, prénom, email (unique), téléphone
Profession	Département, poste, date d'embauche
Compte	Rôle (Admin RH / Employé), Statut (Actif / Inactif)

Fonctionnalités

- ✅ CRUD complet sur les utilisateurs
- ✅ Recherche & filtrage des comptes
- ✅ Réinitialisation de mot de passe
- ✅ Consultation fiche employé détaillée
- ✅ Créer d'autres Admin RH (avec les **mêmes droits**)

⏰ 3. Système de Pointage (Check-in / Check-out)

Flux de la journée

```
Check-in (09:00)
  → Pause matin : début (10:30) / fin (10:45)
  → Pause déjeuner : début (13:00) / fin (14:00)
  → Pause soir : début (16:30) / fin (16:45)
Check-out (18:00)
```

Règles métier strictes

#	Règle
1	**Check-in obligatoire** avant toute pause ou check-out
2	**Un seul** check-in et check-out par jour
3	Une pause doit être **terminée** avant d'en commencer une autre
4	**Une seule pause active** à la fois
5	**Check-out interdit** si une pause est encore active
6	Chaque **type de pause** = **une seule fois** par jour
7	L'employé **ne peut pas modifier** ses pointages historiques
8	Les heures sont enregistrées **automatiquement par le serveur** (pas par le client)

Calculs requis

- **Présence** = Check-out − Check-in
- **Pauses** = Σ(fin − début) pour les 3 pauses
- **Travail effectif** = Présence − Pauses

> NOTE
>
> Exemple donné : Présence 9h, Pauses 1h30, Travail effectif 7h30

📋 4. Gestion des Tâches

Champs d'une tâche

Champ	Détails
Titre	Texte libre
Description	Texte libre
Priorité	`Basse` / `Moyenne` / `Haute`
Statut	`À faire` (défaut) / `En cours` / `Terminée`
Échéance	Date limite
Assignation	Employé assigné

Permissions

- **Admin RH** : CRUD complet + attribution + filtrage par employé/priorité/statut
- **Employé** : Lecture seule + changement de statut uniquement sur ses tâches

> WARNING
>
> L'employé ne peut **ni créer, ni réattribuer, ni modifier le contenu, ni supprimer** une tâche.

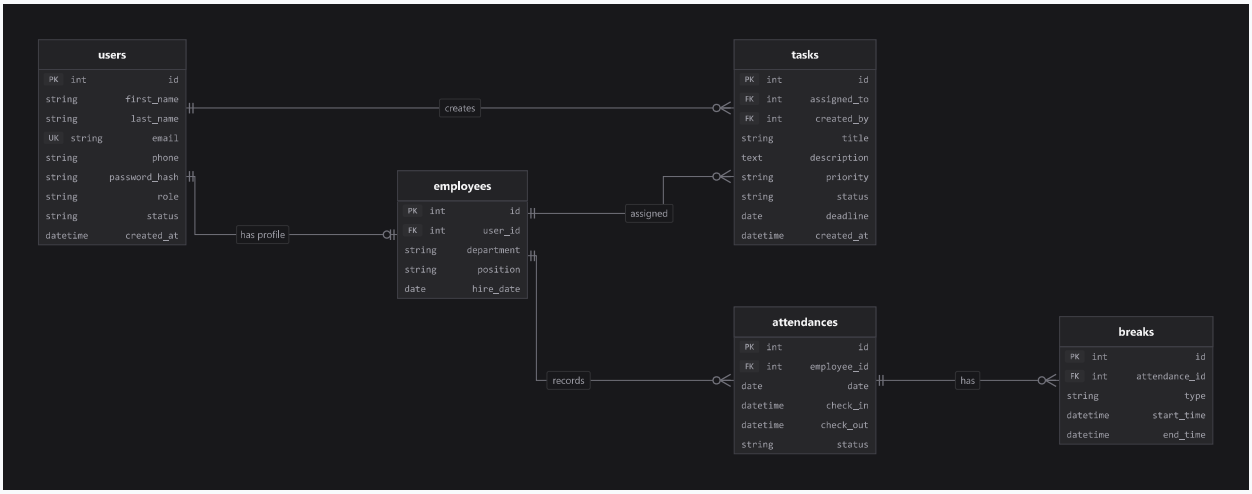
5. Matrice des Permissions Complète

Fonctionnalité	Admin RH	Employé
Connexion / Déconnexion	✅	✅
Créer Admin RH ou Employé	✅	❌
Modifier / Désactiver comptes	✅	❌
Voir tous les utilisateurs	✅	❌
Consulter / MAJ son profil	✅	✅ (limité : téléphone + mdp)
Voir tous les pointages/pauses	✅	❌
Effectuer check-in, pauses, check-out	❌	✅ (personnel)
Voir ses propres pointages	Selon profil	✅
Consulter heures travaillées	Tous les employés	Ses heures uniquement
Déclarer une absence	✅	❌
CRUD tâches	✅	❌
Voir toutes les tâches	✅	❌
Voir ses tâches attribuées	—	✅
Changer statut de ses tâches	—	✅

> [!CAUTION]
> **Sécurité critique** : Modifier un identifiant dans une requête API ne doit **jamais** permettre d'accéder aux données d'un autre employé. Toutes les vérifications **côté backend** obligatoires.

🏗️ 6. Architecture & Base de Données

Tables principales



Stack demandée (dans le cahier des charges)

Couche	Technologie
Frontend	React.js + React Router
Backend	Python + FastAPI + API REST
Base de données	PostgreSQL ou SQLite
Sécurité	JWT + hachage + rôles
Documentation	Swagger/OpenAPI + README
Versioning	Git / GitHub

📊 7. Dashboards

Dashboard Admin RH

- Nombre d'employés **actifs**
- Employés **présents aujourd'hui**

- **Absences** enregistrées
- Nombre total de tâches, **en cours** et **terminées**

Dashboard Employé

- Nom, prénom, département, poste
- Présence du jour + heures de pointage
- Temps de travail effectif
- Compteurs tâches : `À faire` | `En cours` | `Terminées`

📦 8. Livrables attendus

1. Code source frontend + backend dans un **dépôt GitHub**
2. Base de données persistante + **schéma relationnel**
3. Deux interfaces fonctionnelles avec **authentification par rôles**
4. Pointage avec **3 pauses** et **calcul correct des heures**
5. Création, affectation et suivi des **tâches**
6. Documentation API, **README** et instructions de lancement
7. **3 comptes de démo** : 2 Admin RH + 1 Employé

🧠 Concepts clés couverts par ce test

Concept	Ce qu'ils évaluent
JWT Authentication	Login, token generation, token validation, refresh
RBAC (Role-Based Access Control)	Admin vs Employee permissions
Resource Ownership	Un employé ne voit que SES données
Password Hashing	bcrypt/argon2, jamais en clair
Input Validation	Validation côté serveur de toutes les entrées
REST API Design	CRUD endpoints, status codes, error handling
State Machine Logic	Le flux de pointage est une machine à états
Business Rules Engine	Règles complexes sur les pauses, check-in/out
Relational Data Modeling	5 tables avec FK, intégrité référentielle
Frontend Routing & Guards	Protection des routes côté client
Dashboard & Aggregation	Calculs, compteurs, filtres
Responsive UI	Interfaces adaptatives
API Documentation	Swagger auto-generated